ÇANKAYA UNIVERSITY
COMPUTER ENGINEERING DEPARTMENT

CENG 407

Software Requirements Specification

TARAS: Agricultural Analysis & Simulation

ARDA KAHRAMAN - 202111002
HATİCE ELİF ARABACI - 202211008
ÖYKÜ EYLÜL BOZDAĞ - 202311410
MUSTAFA BERK KESKİN - 202211042

TABLE OF CONTENTS

1. Introduction.....2
    1.1. Purpose.....2
    1.2. Aim of the Project.....2
    1.3. Scope.....2
2. General Description.....3
    2.1. Glossary.....3
    2.2. User Characteristics.....5
        2.2.1. Primary User: Farmer.....6
        2.2.2. External Party: Stakeholders.....6
    2.3. Overview of Functional Requirements.....7
    2.4. General Constraints and Assumptions.....7
        2.4.1. Constraints.....7
        2.4.2. Assumptions.....8
3. Specific Requirements.....9
    3.1. Interface Requirements.....9
        3.1.1. Hardware Interface Requirements.....9
        3.1.2. Communication Requirements.....10
        3.1.3. System and Backend Requirements.....12
        3.1.4. Software Interface Requirements.....13
        3.1.5. User Interface Requirements.....14
    3.2. Detailed Description of Functional Requirements.....15
    3.3. Data Preprocessing.....30
        3.3.1. Irrigation Data Preprocessing.....30
        3.3.2. Image Data Preprocessing for Disease Detection.....33
    3.4. Non-Functional Requirements.....35
        3.4.1. Performance Requirements.....35
        3.4.2. Reliability Requirements.....36
        3.4.3. Availability Requirements.....36
        3.4.4. Scalability Requirements.....36
        3.4.5. Security Requirements.....37
        3.4.6. Usability Requirements.....37
        3.4.7. Maintainability Requirements.....37
        3.4.8. Portability Requirements.....37
4. Analysis - UML.....39
    4.1. Use Cases.....39
        4.1.1. Use Case Diagram.....39
        4.1.2. Description of Use Cases.....41
    4.2. Functional Modelling(DFD).....56
        4.2.1. DFD Diagrams.....56
5. Conclusions.....57
6. References.....57

1. Introduction

1.1. Purpose

The purpose of this Software Requirements Specification (SRS) is to define the objectives and functional boundaries of the TARAS precision-agriculture system. TARAS is designed to support farmers in making accurate, data-driven decisions about irrigation, plant health, and overall field management. The system integrates distributed IoT sensors to monitor soil and environmental conditions, applies machine-learning models to generate zone-specific irrigation recommendations, and incorporates computer-vision-based disease assessment to help with the early detection and prevention of crop loss.

This document specifies the requirements for the TARAS mobile application, which provides the primary user interface for real-time field visualization through a digital twin, personalized notifications, decision support, and sustainability tools such as carbon-footprint estimation. The purpose of this SRS is to define the software's goals, intended functions, and constraints so that all stakeholders share a common understanding of the system to be developed.

1.2. Aim of the Project

The aim of our project is to develop an intelligent, integrated precision-agriculture platform that helps farmers make reliable, data-driven decisions about irrigation, plant health, and environmental sustainability. TARAS unifies IoT-based soil and climate sensing, machine-learning models for irrigation prediction, computer-vision systems for early disease detection, and a mobile digital-twin interface that visualizes moisture distribution in real time. By providing actionable recommendations, alerts and clear visual explanations, the system seeks to optimize irrigation efficiency, reduce water and energy waste, support early intervention for plant diseases, and enable farmers to monitor their carbon footprint. Ultimately, the project aims to deliver a low-cost, accessible and adaptive decision-support tool that enhances productivity.

1.3. Scope

The scope of this project includes developing an end-to-end precision-agriculture system that collects soil and environmental data through IoT sensor nodes, transmits them via a LoRa-Raspberry Pi gateway, and stores them in a cloud database. The system processes these data using machine-learning models for irrigation prediction,computer-vision algorithms for disease detection and a large language model. A mobile application provides farmers with a digital-twin visualization of moisture levels, alerts, recommendations and carbon-footprint estimation.

2. General Description

2.1. Glossary

Term	Definition
Farmer	The primary user of TARAS who manages the agricultural field, monitors irrigation, and responds to alerts using the mobile application.
Stakeholder	An organization or representative entity such as an agricultural cooperative, farmer association, or advisory body that uses aggregated and anonymized TARAS outputs for monitoring field performance, supporting farmers, and improving collective decision-making.
Digital Twin	A virtual representation of the physical agricultural field that integrates real-time sensor data, spatial information, and analytical models to simulate field conditions and support decision-making.
Field	The physical agricultural land managed by the farmer, where crops are cultivated and monitored using TARAS.
Field Zone	A distributed embedded device deployed in the field that collects data and transmits it wirelessly to the gateway.
IoT Sensor Node	A central device that aggregates data from multiple IoT sensor nodes and forwards it to the backend system via the internet.
Gateway	A central device that aggregates data from multiple IoT sensor nodes and forwards it to the backend system via the internet.
ESP32	A low-power microcontroller with integrated Wi-Fi and Bluetooth capabilities, used as the main controller in IoT sensor nodes for data acquisition and communication.
Raspberry Pi	A single-board computer used as the system gateway to receive sensor data, perform preliminary processing, and relay information to cloud-based services.
LoRa	A low-power, long-range wireless communication technology operating in the Sub-GHz ISM band, used to transmit sensor data from field nodes to the gateway.
SHT85	A digital temperature and humidity sensor that provides high-accuracy environmental measurements via an I²C communication interface.
Capacitive Soil Moisture Sensor	A sensor that measures soil moisture levels by detecting changes in capacitance caused by water content in the soil,

	offering improved durability over resistive sensors.
Telemetry	The automated process of collecting measurement data from remote sensors and transmitting it to a central system for monitoring and analysis.
Time-Series Data	Data collected sequentially over time intervals, such as sensor readings, used to analyze trends, patterns, and temporal changes in field conditions.
Irrigation Recommendation Engine	A machine-learning–based component that analyzes sensor data, historical records, and crop requirements to generate optimized irrigation suggestions.
Disease Detection Module	A computer-vision component that analyzes leaf images using CNN models to identify plant diseases, estimate severity, and provide early alerts to the farmer.
LLM-based Advisory System	A decision-support component that uses a Large Language Model to generate natural-language guidance for farmers by interpreting sensor data, irrigation recommendations, and disease detection results.
Carbon Footprint	A calculated metric representing the estimated greenhouse gas emissions associated with agricultural activities such as irrigation, fertilizer use, and energy consumption.
REST API	A stateless application programming interface that enables communication between system components using standard HTTP methods and structured data formats such as JSON.
Backend	The server-side components of the TARAS system responsible for data storage, processing, analytics, and integration with machine-learning models.
Frontend	The user-facing components of the TARAS system, including the mobile application interface through which users view data, receive alerts, and interact with recommendations.
PostgreSQL	An open-source, ACID-compliant relational database management system that supports advanced features such as JSONB, extensions, strong indexing, and complex queries, making it suitable for scalable and data-intensive applications.
TimescaleDB	A PostgreSQL extension optimized for time-series data. It uses hypertables to efficiently store, query, and compress IoT sensor readings.
PostGIS	A spatial database extension for PostgreSQL that adds support for geographic objects, enabling storage and analysis of field zones, polygons, sensor locations, and other geospatial data

	used in the digital twin of the farm.
CNN(Convolutional Neural Network)	A deep-learning model designed for image analysis, used in this project to detect plant diseases by classifying leaf images and identifying visual patterns of stress or infection.
ML Model	A trained computational model that learns patterns from data and generates predictions or classifications, such as irrigation needs or disease likelihood.
Inference	The process of applying a trained machine-learning model to new input data to generate predictions or outputs.
Store-and-Forward	A communication strategy in which data is temporarily stored locally when connectivity is unavailable and transmitted once the connection is restored.
Deep Sleep Mode	A low-power operating state of microcontrollers in which most system functions are disabled to conserve energy between data collection cycles.
JWT(JSON Web Token)	A secure, compact token format used for authentication and authorization between the frontend and backend components of the system.
Telemetry Packet	A structured data unit containing sensor readings, timestamps, and device identifiers transmitted from sensor nodes to the gateway or backend.
Threshold	A predefined limit or value that, when exceeded or violated, triggers system alerts or recommendations.
Normal Operating Conditions	Conditions in which the system operates with stable hardware, expected network connectivity, and no major external disruptions such as prolonged power outages or hardware failures.
Guidance Session	A short onboarding or instructional interaction designed to help users understand basic system functionality.

2.2. User Characteristics

The system is designed with two key user groups in mind: the primary user (farmer), and external institutional stakeholders (such as cooperatives or environmental reporting bodies). Each group has distinct abilities, responsibilities, and interaction needs, which shape the system's interface and functionality.

2.2.1. Primary User: Farmer

Profile: Small-to-medium-scale farmers who rely on their experience for regular irrigation and crop-health decisions. They interact with the system mostly in outdoor field conditions.

Age Range: Typically 20–65, though many may be older and accustomed to traditional farming practices.

Technical Skills: Low-to-moderate comfort with digital tools. Interfaces shall be simple, intuitive, and optimized for mobile use, with minimal steps and clear visual cues.

Work Environment: Outdoor, field-based conditions with limited connectivity, variable lighting, and frequent movement.

Key Characteristics:

- Relies heavily on intuitive decision-making
- Faces challenges like water scarcity, plant disease uncertainty
- Needs clear, non-technical explanations rather than complex analytics
- Requires solutions that are low-cost, reliable and field-tolerant

2.2.2. External Party: Stakeholders

Profile: Stakeholders are organizations that have access to the system but with **highly restricted, read-only permissions**. They do not participate in daily decision-making, sensor management, or farm operations. Their role is limited to viewing summaries and sustainability-related metrics for monitoring or reporting purposes.

Examples:

- Agricultural cooperatives
- Sustainability and environmental monitoring institutions
- Government programs that track water usage, efficiency, or carbon impact

Interaction Model: Stakeholders log into a **simplified, summary-focused dashboard** that displays high-level insights such as water consumption trends, irrigation efficiency metrics, carbon-footprint estimates, and sustainability indicators. They don't upload data, adjust system settings, or interact with field-level features such as disease detection or digital-twin visualizations. Their interaction is passive and observational, they only consume the system's output and do not influence or control the platform's internal processes.

2.3. Overview of Functional Requirements

TARAS provides an integrated set of functional capabilities to support data-driven decision-making in smart agriculture. At its core, the system continuously collects soil data from distributed IoT sensor nodes deployed in the field. These data are transmitted through a long-range communication infrastructure to a central gateway and forwarded to the cloud backend, where they are stored as time-series records and made available for analysis.

Based on the collected data, TARAS performs analytical and decision-support functions, including machine-learning–based irrigation recommendations and computer-vision–based plant disease detection.

The mobile application serves as the primary interaction tool for all users to visualize real-time and historical field conditions through a digital twin, receive recommendations, upload plant images, analyze leaf health, and interact with advisory features. In addition, the system supports sustainability-related functions such as carbon footprint calculation when farmers input necessary information. Also external stakeholders can view summarized field data and sustainability information for monitoring and sustainability assessment purposes.

Together, these functional requirements define an end-to-end system that integrates sensing, communication, data processing, analytics, visualization, and user interaction to deliver reliable and actionable agricultural decision support.

2.4. General Constraints and Assumptions

2.4.1. Constraints

- **Hardware Constraints:**
  The sensor nodes operate on ESP32-C6 microcontrollers with limited RAM and processing capability, restricting on-device computation and shifting most analytical tasks to the gateway(Raspberry Pi) or cloud.

  All soil-moisture, temperature, and humidity sensors must operate within strict voltage ranges (typically 3.0–3.6 V), limiting hardware choices and influencing the power-management design. Since all sensor nodes are battery-powered, energy consumption must remain minimal to ensure long-term field deployment.

- **Communication Constraints:**
  LoRa provides long-range but low-bandwidth communication and therefore, sensor data packets must be compact and optimized for transmission.

Rural agricultural settings often experience weak or unstable connectivity, which requires store-and-forward mechanisms and causes unavoidable delays in synchronization with the cloud backend.

- **Environmental Constraints:**
  Devices that are all field-deployed must withstand harsh outdoor conditions such as humidity, temperature extremes, rainfall, dust, and direct sunlight.

  Variations in soil composition and microclimatic differences introduce noise into sensor readings, potentially affecting predictive model accuracy.

- **Software & Data Constraints:**
  Available training data for machine-learning models is limited and may not represent all crop types or environmental contexts, reducing early-stage model accuracy.

  Real-time inference must meet latency requirements appropriate for on-field decision-making, which imposes limits on model size and computational complexity.

- **Operational Constraints:**
  Continuous internet connectivity cannot be guaranteed for either the gateway or the user's mobile device, requiring offline-tolerant system behavior.

  The mobile application must support a wide range of user devices, specifically Android versions 7.0–16.0 and iOS versions 15.1–26.1.

- **Budget & Resource Constraints:**
  High-quality agricultural sensors, LoRa modules, and gateway hardware incur significant cost constraints.

  Real-world field access for testing may be limited, which may reduce the frequency and duration of validation cycles.

2.4.2. Assumptions

- **Sensor Deployment Assumptions:**
  Sensors are assumed to be properly installed, calibrated, and positioned in pre-determined areas of the field.

  Battery-powered sensor nodes are assumed to maintain sufficient charge throughout their scheduled data collection intervals.

- **Communication Assumptions:**
  The LoRa gateway and sensor nodes are assumed to remain within effective communication range under normal operating conditions.

  Temporary connectivity outages are expected. It is assumed that store-and-forward mechanisms will eventually synchronize all buffered data without loss.

- **User Behavior Assumptions:**
  Farmers are assumed to regularly monitor the mobile application and take appropriate actions based on irrigation recommendations or disease analysis results.

  External stakeholders are assumed to require only high-level, read-only access for monitoring and reporting purposes.

- **Machine-Learning Assumptions:**
  ML models assume that input data (soil moisture, temperature, humidity, irrigation history) remains aligned with previously observed patterns.

  The irrigation recommendation engine assumes that environmental anomalies (e.g., extreme weather, pump malfunctions) are infrequent and will be reflected in sensor readings.

- **Mobile Application Assumptions:**
  The mobile application assumes intermittent but recurring Internet connectivity to retrieve sensor data, digital twin updates, and ML-driven recommendations.

3. Specific Requirements

3.1. Interface Requirements

3.1.1. Hardware Interface Requirements

The TARAS system interacts with a distributed network of **Io**T sensor nodes, a long-range communication module, and a **Raspberry Pi–based gateway**, which requires consistent electrical, communicational, and operational compatibility across all hardware components. Each sensor node incorporates an ESP32-C6 microcontroller, which provides the necessary GPIO pins, I²C bus capabilities, and low-power operational

modes, which are needed for continuous agricultural monitoring[1]. The hardware interface must support stable connections to capacitive soil-moisture sensors, SHT85 temperature-humidity units, and additional digital sensors through standardized I²C communication. All sensors must operate within the microcontroller's voltage range (3.0–3.6 V) and support uninterrupted periodic sampling[1][2].

The **wireless communication interface** is based on **LoRa** transceivers operating in the Sub-GHz ISM band[3]. Every sensor node must be electrically and logically compatible with the LoRa E32-900 or equivalent modules, using UART-based communication with defined baud rates, buffer sizes, and timing requirements. The system must ensure that transmitted packets conform to the gateway's LoRa configuration parameters (frequency, spreading factor, bandwidth, and coding rate) to guarantee long-range communication in rural field conditions. Placement of antenna and power levels must be maintained within regulatory limits.

The **gateway** hardware, built on a **Raspberry Pi**, must support direct UART or SPI interfacing with the LoRa module and maintain stable Ethernet or Wi-Fi connectivity to the cloud backend. The Raspberry Pi must provide sufficient processing capability to buffer, preprocess, and forward sensor data during **network outages**, and it must ensure compatibility with external storage for local logging. Its USB and GPIO interfaces must remain available for optional actuators, calibration tools, or expansion modules that may be incorporated during future system development.

Power interfaces are also fundamental. Each sensor node must operate reliably on 3.7 V Li-Ion or Li-Po battery systems, regulated to safe microcontroller and sensor input levels. The hardware must support energy-saving modes and deep sleep states, without causing voltage instability. All components must be physically robust and weather-resistant to withstand outdoor agricultural deployment. The complete hardware interface architecture must guarantee **stable electrical operation**, **reliable communication**, and **long-term durability** while supporting seamless data acquisition and real-time integration with the TARAS software ecosystem.

3.1.2. Communication Requirements

The communication infrastructure is built on a multi-layered architecture designed to ensure reliable, energy-efficient, and fault-tolerant data transfer across all components of the TARAS system. At the field level, distributed ESP32-C6 sensor nodes communicate with soil-moisture and

environmental sensors locally, then transmit telemetry data to a central Raspberry Pi gateway using The TARAS system's data infrastructure is built on a scalable and reliable architecture designed to support continuous IoT telemetry streaming, real-time machine learning inferences, a disease detection module and mobile application interactions. The system's database will be created with PostgreSQL, and it will be hosted using a cloud service such as AWS, enabling the system to efficiently manage large-scale agricultural fields, enabling seamless historical analysis, predictive irrigation calculations, and model calibration processes.

TARAS's backend architecture will consist of two layers: the cloud-based main backend and the service layer that manages mobile clients. The main backend consists of API services running on a commercial cloud hosting service such as AWS. Sensor data from the gateway first reaches the API layer, where it undergoes formatting before being transferred to time series tables. The backend service provides all the necessary API endpoints for users for them to view the digital twin map, receive irrigation recommendations, upload images for disease detection and receive system notifications. This layer will include caching strategies to ensure devices can operate in intermittent internet conditions. Authentication will be provided via industry standard solutions such as AWS Cognito, which ensures two-way security. The data model includes a structured schema consisting of sensor nodes, time-series telemetry records, field regions, digital twin snapshots, irrigation recommendation processing history, disease detection outputs and user profiles. All tables are designed to maintain ACID integrity, and relationship constraints, identification mechanisms and data validation rules will be enforced. Image files used for disease detection will be stored in cloud storage, not directly in the database; only the file path and classification outputs will be stored in the database.

The system will be using LoRa transceivers operating in the Sub-GHz ISM band. Each node interfaces with its LoRa module via UART, enabling low-power, long-range wireless transmission that remains stable across wide agricultural areas without requiring wired infrastructure. This design allows sensor nodes to operate for extended periods on battery power.

To ensure continuity and data integrity in rural environments, where network interruptions are common, the system employs a multi-tiered store-and-forward mechanism. Both the ESP32 nodes and the Raspberry Pi gateway maintain local caches for temporarily storing measurements. If a LoRa transmission attempt fails, or if the gateway can’t reach the cloud backend, data is buffered locally and automatically

synchronized once connectivity is restored. LoRa-level acknowledgments and gateway-side packet validations ensure that no telemetry is lost and that all forwarded data meets integrity and completeness requirements.

The Raspberry Pi communicates with the backend through secure HTTP or MQTT-based APIs over Ethernet or Wi-Fi. The gateway formats validated packets into standardized message structures and transmits them over encrypted channels like HTTPS, which provides reliable integration with database and analytics services.

This communication architecture ensures robust, secure and energy-efficient end-to-end data flow from field sensors to cloud intelligence. It also maintains operational resilience under the constraints of remote agricultural environments .

3.1.3. System and Backend Requirements

The TARAS system's data infrastructure is built on a scalable and reliable architecture designed to support continuous IoT telemetry streaming, real-time machine learning inferences, a disease detection module, and mobile application interactions. The system's core database will be implemented using PostgreSQL, providing a robust relational foundation for structured agricultural data management[4].

To efficiently handle sensor telemetry, the architecture supports the use of the TimescaleDB extension, which may be enabled to improve performance for time-based queries, historical analysis, and aggregation-heavy workloads[5]. When utilized, this extension facilitates efficient storage, partitioning, and compression of sensor data, enabling scalable management of large agricultural fields and supporting predictive irrigation calculations, digital twin visualization, and machine learning model calibration processes.

When spatial field analysis is required, the PostGIS extension can be enabled to support geospatial queries and region-based operations, ensuring seamless integration with the digital twin component[6].

TARAS's backend architecture consists of two logical layers: an AWS-based main backend and a service layer responsible for managing mobile client interactions. The main backend includes API services deployed on AWS and a PostgreSQL database hosted on AWS.Sensor data collected from field gateways is first received by the API layer, where it undergoes validation and formatting before being persisted in the database.

The TARAS backend architecture will utilize Node.js as the server-side runtime environment for managing RESTful APIs, processing incoming sensor data, and coordinating interactions between the mobile application, database services, and machine-learning modules.

The backend exposes RESTful API endpoints that allow users to access digital twin maps, receive irrigation recommendations, upload plant images for disease detection, and receive system notifications.

3.1.4. Software Interface Requirements

The TARAS software architecture relies on a set of coordinated interfaces that connect the machine-learning components, backend services and mobile application. These interfaces ensure that irrigation prediction, disease diagnosis, and the LLM-based advisory system operate consistently. The system's ML modules communicate with the backend through well-defined API contracts, enabling the exchange of structured sensor data, user input, and model outputs.

The irrigation-prediction module requires a time-series interface with the cloud database. The backend provides the ML component with aggregated sensor records; soil moisture, temperature, humidity, and historical irrigation events. The ML model returns a prediction object that includes the recommended irrigation amount and auxiliary variables used for explainability. The interface supports synchronous requests from the mobile application as well as scheduled, automated evaluations triggered by new field data arriving from the gateway.

The disease-detection component interacts with the system through an image-processing interface exposed by the backend. The mobile application uploads leaf images in compressed JPEG or PNG format, and the backend forwards these images to the CNN inference engine. The model returns a classification label, confidence score, and an explanation map where available. The software interface ensures consistent data encoding, maximum file size constraints, and predictable inference times so that the mobile client receives results without noticeable delay.

The LLM-based advisory module operates on top of the system's analytical outputs and requires a specialized interface to retrieve the latest sensor data, irrigation predictions, and disease-diagnosis results. The backend provides the model with a unified context package containing structured environmental data, recent recommendations, and user-submitted information such as irrigation history or fertilizer

application. The LLM returns a natural-language advisory response, including explanations, warnings, or scenario-based comparisons. This interface allows the mobile application to request contextualized advice in real time while ensuring that the LLM's outputs remain consistent with the most recent sensor readings and model predictions.

The mobile interface uses the designated API endpoints to retrieve ML results, send image data, and request advisory messages. All interfaces must maintain compatibility with the PostgreSQL-backed data layer and must handle synchronization between the gateway, cloud, and mobile application without data loss. These software interfaces also ensure that the ML-driven components such as irrigation prediction, disease detection, and LLM advisory, integrate seamlessly with the user-facing mobile application and the backend infrastructure.

3.1.5. User Interface Requirements

The TARAS mobile application serves as the primary interaction layer between the farmer and the underlying sensing, prediction and visualization ecosystem, requiring a cohesive and intuitive interface that enables efficient decision-making under varying environmental conditions.

Overall, the user interface must maintain responsiveness across diverse screen sizes, uphold visual consistency aligned with the system's design identity and model outputs into actionable, farmer-friendly insights that support reliable, data-driven agricultural management.

The user interface is structured around a persistent navigation bar that provides direct switching between tabs that will enable the user to access their desired function such as Overview, Timetable, Disease Detection, and Settings. A global floating button remains visible across these primary screens to engage the AI assistant overlay. This conversational interface processes natural language queries and enables users to execute "What-if" scenario simulations, providing predictive insights regarding field management directly within the application.

Access to the application is managed through a role-based login interface that authenticates distinct user types, including farmers and stakeholders, and accordingly adjusting interface permissions and data visibility according to the user's specific level of access.

The central dashboard functions as a digital twin, rendering an interactive 2D/3D visualization of the field. This model maps real-time sensor inputs, specifically soil moisture and temperature, to color-coded zones within the canvas. Users navigate this digital representation by

dragging to rotate the perspective and tapping specific sectors to isolate detailed data. To support immediate decision-making, the visualization automatically highlights zones where readings fall outside optimal ranges, providing visual alerts for conditions such as drought stress or over-irrigation.

The **disease detection module** utilizes a live camera viewfinder, enabling users to capture plant imagery for on-screen classification of disease types and severity. Additionally, the specific timetable interface presents a schedule of irrigation and fertilization tasks, while also providing input fields for logging fuel, energy, and fertilizer consumption to support **carbon footprint calculations**.

Navigation shall remain simple and consistent across Android and iOS devices, using a navigation bar to provide fast access to primary screens such as Dashboard, Digital Twin, Disease Detection, and Settings. The interface must also deliver concise push notifications for irrigation needs, sensor anomalies, disease risks, and connectivity issues, ensuring timely awareness.

3.2. Detailed Description of Functional Requirements

- **Microcontroller - Sensor Interface**

Name	**Microcontroller - Sensor Interface**
Purpose / Description	The sensor interface enables reliable acquisition of soil moisture, temperature, humidity, and related environmental data from IoT sensor nodes. Each ESP32-C6 node reads SHT85 sensors via I²C and the soil-moisture probe via ADC. The interface ensures proper electrical levels (3.0–3.6 V), periodic sampling, and noise-resistant communication to support continuous outdoor field monitoring.
Inputs	● I²C readings from SHT85 temperature-humidity sensor. ● ADC readings from capacitive soil-moisture sensor. ● I²C/ADC configuration parameters (address, bus speed, ADC resolution, sampling rate). ● Regulated 3.0-3.6 V power input.

Name	Microcontroller - Sensor Interface
Processing	1. Initialize I²C bus and ADC at boot. 2. Perform periodic sampling of all sensors. 3. Verify sensor presence via I²C acknowledgment. 4. Apply calibration and scaling to raw readings. 5. Filter noisy values using smoothing/averaging. 6. Format processed data into LoRa-compatible packets. 7. Timestamp each measurement for synchronization.
Outputs	● Validated sensor packets (soil moisture, temperature, humidity). ● LoRa-formatted payloads for transmission. ● Sensor status indicators (active, missing or calibrated).
Error Handling	● Retry I²C communication on missing acknowledgment. ● Mark and suppress physically invalid readings. ● Reset I²C bus after collisions/noise. ● Suspend sampling under low-voltage conditions. ● Log diagnostic flags for gateway-side monitoring.

- **Gateway Hardware Interface Functional Requirement**

Name	Gateway Hardware Interface
Purpose / Description	The gateway hardware interface enables reliable reception, preprocessing, buffering, and cloud-forwarding of sensor data from LoRa-enabled nodes. A Raspberry Pi communicates with the LoRa transceiver via UART and uses Ethernet/Wi-Fi for cloud upload, which provides local storage to maintain operation during network outages.
Inputs	● LoRa packets received via UART/USB transceiver. ● Serial port and LoRa configuration parameters. ● System time for packet timestamping. ● Ethernet/Wi-Fi connectivity for cloud upload. ● Local storage for offline buffering.
Processing	1. Initialize serial communication with LoRa module. 2. Listen for incoming LoRa frames on the serial interface. 3. Decode and validate packet headers, checksum, and payload. 4. Timestamp and format data for cloud transmission. 5. Buffer packets locally when offline; sync once reconnected. 6. Forward validated data to cloud via HTTP/MQTT.

Name	Gateway Hardware Interface
Outputs	● Cloud-delivered sensor data(JSON or binary). ● Diagnostic logs (success rates, hardware status). ● Locally stored backup files for offline periods. ● Optional ACK/config messages to nodes.
Error Handling	● Retry serial initialization on failure. ● Discard corrupted packets and log integrity errors. ● Queue data locally when cloud upload fails; retry periodically. ● Trigger storage alerts or overwrite oldest files if full. ● Reset LoRa interface if module becomes unresponsive.

- **Power Interface Functional Requirement**

Name	Power Interface
Purpose / Description	The power interface ensures stable and efficient energy delivery to ESP32-C6 nodes, sensors and the LoRa module using a 3.7 V Li-Ion/Li-Po battery regulated to safe operating levels. It supports low-power modes, deep sleep operation and voltage stability to maximize battery life. The interface also guarantees reliability in outdoor conditions by preventing undervoltage, brownouts and power-related data loss.
Inputs	● 3.7 V Li-Ion/Li-Po battery input. ● Regulated 3.0–3.6 V output for ESP32-C6 and sensors. ● Power-management configuration (sleep intervals, wake triggers). ● Voltage and current readings from onboard regulators.
Processing	1. Regulate battery voltage to component-safe levels. 2. Manage deep sleep and low-power modes. 3. Monitor voltage stability to prevent brownouts. 4. Allocate power to sensors and LoRa module based on sampling/transmission cycles. 5. Log power events (sleep, wake, low voltage).
Outputs	● Stable power delivery to microcontroller, sensors and LoRa module. ● Power-state indicators (active, sleep, low voltage). ● Logged power-performance data for diagnostics.
Error Handling	● Enter safe mode on undervoltage conditions. ● Suspend sensing/transmission when power is unstable.

Name	Power Interface
	● Trigger automatic restart if brownout occurs. ● Log critical power faults for gateway diagnostics.

- **Environmental / Enclosure Interface Functional Requirement**

Name	Environmental / Enclosure
Purpose / Description	The environmental interface ensures that all hardware components operate reliably in outdoor conditions by providing protection against moisture, dust, UV exposure, temperature variation and mechanical stress. Weather-resistant enclosures, sealed connectors, and durable materials maintain sensor and node integrity during long-term deployment.
Inputs	● Outdoor environmental exposure (rain, dust, UV, temperature changes). ● Mechanical stress from handling or field conditions. ● Moisture and particulate ingress risks. ● Thermal load from sun exposure.
Processing	1. Seal enclosures to prevent water and dust ingress. 2. Regulate airflow or thermal dissipation to maintain safe operating temperature. 3. Shield sensors and electronics using protective casing and strain-relieved cabling. 4. Prevent corrosion through coated PCBs or protected connectors.
Outputs	● Stable device operation under outdoor conditions. ● Protected internal electronics and sensors. ● Maintained enclosure integrity and mechanical stability. ● Environmental health status indicators.
Error Handling	● Detects abnormal temperature/humidity inside enclosure. ● Trigger warnings or reduced operation if thermal or moisture limits are exceeded. ● Log environmental fault conditions for maintenance. ● Enter protective mode or shut down to prevent hardware damage.

- **LoRa Uplink Communication Functional Requirement**

Name	**LoRa Uplink Communication**
Purpose / Description	Provides long-range, low-power wireless transmission from ESP32-C6 sensor nodes to the Raspberry Pi gateway using LoRa in the Sub-GHz ISM band. Ensures reliable UART-based communication between the node and the LoRa module for stable telemetry delivery across large agricultural areas.
Inputs	● Sensor payloads encoded by the ESP32-C6. ● UART data transmitted to the LoRa module. ● LoRa configuration parameters (frequency, SF, bandwidth, coding rate, TX power). ● Node ID and packet header/checksum information.
Processing	1. Initialize UART–LoRa interface at node startup. 2. Package sensor data into LoRa frames (header, payload, checksum). 3. Transmit packets periodically or event-driven via LoRa RF link. 4. Gateway receives and decodes incoming LoRa frames. 5. Validate packets and extract payload content. 6. Optionally read RSSI/SNR for link-quality monitoring.
Outputs	● Successfully received and validated sensor data at the gateway. ● Transmission status indicators (success, retries, and etc). ● Parsed payloads ready for buffering or cloud upload.
ErrorHandling	● Retry transmission on UART or RF failure. ● Discard corrupted packets and attempt retransmission. ● Reset UART interface on buffer overflow or communication timeout. ● Switch to local buffering if gateway acknowledgments are not received. ● Log link-quality and transmission errors for diagnostics.

- **Store-and-Forward Communication Reliability Functional Requirement**

Name	**Store-and-Forward Communication Reliability**
Purpose / Description	Ensures reliable data delivery in environments with intermittent connectivity by implementing local caching at both sensor

Name	Store-and-Forward Communication Reliability
	nodes and the Raspberry Pi gateway. When LoRa or internet transmission fails, data is stored temporarily and automatically synchronized once the link becomes available, preventing telemetry loss.
Inputs	● Sensor data awaiting transmission. ● LoRa link status between node and gateway. ● Internet connectivity status at the gateway. ● Local storage resources (RAM/flash on node, filesystem on gateway). ● Packet metadata (timestamps, sequence numbers).
Processing	1. Detect failed LoRa or cloud transmission attempts. 2. Store unsent data in local buffers. 3. Periodically check link availability for retransmission. 4. Synchronize buffered packets in chronological order once connectivity returns. 5. Verify acknowledgment from the gateway or cloud before deleting local copies. 6. Filter duplicates and ensure packet ordering via timestamps or sequence indicators.
Outputs	● Lossless delivery of all sensor telemetry to the gateway and cloud. ● Updated synchronization logs showing queued and transmitted packet counts. ● Stable, continuous data flow despite connectivity interruptions.
Error Handling	● Apply retry/backoff when repeated transmission failures occur. ● Manage buffer overflow by overwriting oldest data or issuing warnings. ● Log cache write/read errors for diagnostics. ● Prevent data corruption by validating stored packets before retransmission.

- **Gateway-to-Cloud Communication Functional Requirement**

Name	Gateway-to-Cloud Communication
Purpose / Description	Enables secure, reliable transmission of validated sensor data from the Raspberry Pi gateway to the cloud backend using

Name	Gateway-to-Cloud Communication
	HTTP/MQTT protocols. Ensures encrypted communication, standardized payload formatting and uninterrupted data flow even in the presence of intermittent internet connectivity.
Inputs	● Validated sensor packets received from LoRa uplink. ● Cloud endpoint URLs and authentication parameters. ● Ethernet or Wi-Fi network connection status. ● Protocol configuration ● System timestamp for message scheduling.
Processing	1. Format decoded sensor data into standardized JSON or binary structures. 2. Establish secure communication using TLS/HTTPS. 3. Publish or upload messages via MQTT or HTTP POST. 4. Monitor server responses (ACK, status codes) to verify delivery. 5. Cache unsent data locally during connectivity loss. 6. Automatically retransmit buffered data once the network connection is restored. 7. Log communication events, including latency/error codes.
Outputs	● Successfully transmitted sensor data stored in the cloud. ● Diagnostic logs detailing transmission success, failures and response codes. ● Updated local buffer status reflecting synchronized and pending data.
Error Handling	● Retry failed HTTP/MQTT transmissions using backoff or scheduled retries. ● Cache packets locally when there's no internet. ● Handle TLS handshake failures by suspending upload until a secure channel is available. ● Discard or reformat invalid payloads before retransmission. ● Log all communication errors for remote diagnostics.

- **Sensor Telemetry Storage and Management Functional Requirement**

Name	Sensor Telemetry Storage and Management
Purpose / Description	Defines the system's responsibility to receive, store, manage, and aggregate high-frequency sensor telemetry data in a time-series-oriented data infrastructure. The system shall ensure that collected sensor measurements can be reliably accessed

Name	Sensor Telemetry Storage and Management
	and processed to support irrigation prediction, digital twin visualization and historical trend analysis.
Inputs	● Incoming telemetry packets from gateway: ○ soil_moisture ○ temperature ○ humidity ○ node_id ○ timestamp
Processing	1. Validate the structure and data types of incoming telemetry. 2. Automatically partition data by time and node identifier. 3. Apply compression policies to historical data chunks. 4. Generate continuous aggregates, including: ○ hourly soil moisture averages, ○ daily moisture trends, ○ temperature–humidity correlation metrics. 5. Expose optimized query interfaces for ml modules and digital twin services.
Outputs	● Telemetry records accessible through REST APIs. ● Summarized datasets for UI rendering, ML processing. ● Historical trend datasets for analytics and visualization.
Error Handling	● If a schema mismatch occurs, the telemetry record is marked as invalid and logged. ● If the database connection is unavailable, records are temporarily queued at the gateway and forwarded once connectivity is restored. ● If hypertable retention thresholds are reached, older records are removed according to the defined data-retention policy.

- **Disease Detection Database Recording Functional Requirement**

Name	Disease Detection Database Recording
Purpose / Description	Stores the output of the disease detection module; including image path, classification, severity, confidence, and timestamp, inside a structured disease results table in PostgreSQL.

Name	**Disease Detection Database Recording**
Inputs	● Classification result from CNN model ● Cloud storage URL of user-uploaded image ● User ID (farmer) ● Device and field metadata
Processing	1. Validate disease result schema. 2. Insert record into disease results table. 3. Update disease statistics for the field. 4. Trigger notification system if severe disease detected.
Outputs	● Database record (row) created. ● Disease history accessible via API. ● Push notification sent.
Error Handling	● If the database write operation fails, the system performs an automatic retry before reporting an error. ● If the classification output is missing, the system records the entry with the label “unidentified.”

- **Digital Twin Data Assembly Functional Requirement**

Name	**Digital Twin Data Assembly**
Purpose / Description	Supports the Digital Twin by managing field geometry, zone polygons, and spatial queries using PostgreSQL + PostGIS.
Inputs	● Field boundary polygons ● Zone definitions (polygons) ● Sensor-node geolocation coordinates ● Latest moisture/temperature telemetry
Processing	1. Store and index spatial data 2. Map telemetry to nearest field zone 3. Generate zone-level moisture/temperature aggregates 4. Prepare structured datasets for heatmap rendering
Outputs	● Zone-level moisture distribution ● 2D/3D Digital Twin datasets ● Spatial queries for crop/zone analysis
Error Handling	● If a geometry record is invalid, it’s rejected and logged for review.

Name	Digital Twin Data Assembly
	● If telemetry data arrives without a spatial reference, it is assigned to an “unmapped” zone. ● If a spatial index becomes corrupted, the system rebuilds the index to restore proper query performance.

- **Disease Detection Functional Requirement**

Name	Disease Detection Functional Requirement
Purpose / Description	Enables users to upload plant leaf images and receive disease classification using the system’s CNN-based image-processing module. The module analyzes uploaded images, detects disease type/severity, generates confidence scores, and provides actionable recommendations.
Inputs	● User-uploaded plant leaf image. ● Optional metadata: crop type, field zone, timestamp. ● Pretrained CNN weights and preprocessing parameters. ● Stored disease classes in the database.
Processing	1. Validate image size, format, and resolution. 2. Preprocess image (resize, normalize, color correction). 3. Run CNN inference to classify disease type & severity. 4. Store classification output into PostgreSQL (disease table) with image path (cloud storage). 5. Forward result to mobile application for visualization.

Outputs	● Disease label
	● Severity score.
	● Confidence value.
	● API response to the mobile client.
	● Database entry containing:
	  ○ image_url
	  ○ classification
	  ○ confidence
	  ○ severity
	  ○ timestamp

Error Handling	● If the image format is invalid, the system returns an “unsupported file format” error.

Name	Disease Detection Functional Requirement
	● If the image file is corrupted, the system rejects the upload and records it in the logs. ● If the inference engine is unavailable, the system attempts a retry and then returns a error if the issue persists. ● If inserting the result into the database fails, the system logs the failure and returns a partial-success response to the client. ● If metadata is missing, the system proceeds using default assumptions defined by the backend.

- **Irrigation Prediction Request & Response Handling Functional Requirement**

Name	Irrigation Prediction Request & Response
Purpose / Description	Generates irrigation recommendations based on aggregated sensor data, historical irrigation logs, and environmental patterns using the ML prediction module.
Inputs	● Soil moisture, humidity, temperature ● Field metadata (crop type, soil type) ● Historical irrigation events
Processing	1. Retrieve latest time-series data from PostgreSQL 2. Validate data completeness and imputation rules 3. Feed processed dataset into irrigation ML model 4. Generate recommended irrigation amount & priority level
Outputs	● JSON recommendation object ● Explanation variables(feature importance, moisture deficits)
Error Handling	● If missing sensor data = fallback to last-known valid sample ● If ML model fails = notify backend & return “unavailable” ● If database unavailable = queue prediction job

- **Role-Based Access Control & IU Authorization Functional Requirement**

Name	**Role-Based Access Control & UI Authorization**
Purpose/Description	Defines role-based access rules for the TARAS mobile and web interfaces. The system controls which screens, features, and data are visible or accessible to users based on their assigned role, ensuring secure and role-appropriate interaction with the application.
Inputs	● Authenticated User Session ● User Role ● Requested UI Screen or Feature ● Navigation Events
Processing	1. Role Identification: Determine the user's role after successful authentication. 2. Access Evaluation: Check requested screen or feature against role-based permission rules. 3. UI Rendering: Display only the interface components permitted for the user's role. 4. Navigation Filtering: Hide or disable navigation items that are not allowed for the current role. 5. Session Enforcement: Enforce access rules during navigation and screen transitions.
Outputs	● Role-appropriate UI layout. ● Accessible navigation menu based on user role. ● Authorized access to data, visualizations and actions.
Error Handling	● If a user attempts to access a restricted screen, redirect to an authorized default screen. ● If role information is missing or invalid, terminate the session and require re-authentication. ● If permission rules fail to load, restrict access to read-only or safe default views.

- **Accessibility & Inclusive UI Support Functional Requirement**

Name	**Accessibility & Inclusive UI Support**
Purpose / Description	Defines accessibility features of the TARAS user interface to ensure that the mobile application is usable by farmers and stakeholders with varying physical, visual, and cognitive abilities. The interface prioritizes clarity, readability, and ease of interaction under field conditions.

Name	Accessibility & Inclusive UI Support
Inputs	● System Accessibility Settings (Font Size, Contrast) ● Device Accessibility Preferences ● User Interaction Events ● UI Rendering Context
Processing	1. Font Scaling: Adapt text size according to system-level font and accessibility settings. 2. Contrast Enforcement: The system shall ensure a minimum contrast ratio of 4.5:1 between text and background for all primary information elements displayed in the interface[7]. 3. The system shall ensure that all interactive UI elements have a minimum touch target size that was stated in Web Content Accessibility Guidelines[8]. 4. Visual Simplification: The system shall limit the number of primary interactive elements displayed on a single screen to a maximum of five, prioritizing critical actions and information. 5. Consistent Feedback: The system shall provide immediate visual feedback for user interactions, including pressed, selected, loading, and error states.
Outputs	● Accessible UI layout with readable text and clear contrast. ● Enlarged and touch-friendly interactive elements. ● Simplified screens optimized for field usage.
Error Handling	● If accessibility preferences cannot be applied, load the default high-contrast and large-text layout. ● If UI rendering fails due to accessibility overrides, reset to a safe default layout. ● If device accessibility settings change during runtime, re-render the UI to reflect updated preferences.

- **Mobile Application Framework & Navigation Functional Requirement**

Name	Mobile Application Framework & Navigation
Purpose / Description	Defines the initialization, localization and navigation structure of the TARAS application. It establishes the primary user interface shell, ensuring consistent navigation and language support across Android and iOS devices.
Inputs	● System Language Settings (Locale) ● System Theme Preference (Light/Dark)

Name	Mobile Application Framework & Navigation
	● User Credentials (Login/Skip) ● Touch Interaction Events
Processing	1. Localization: Detect device system language on startup apply Turkish(TR) by default or English(EN) if explicitly set. 2. Theme Application: Apply the color schema (Light/Dark) based on system settings or user override. 3. Authentication Routing: Direct user to Login Screen or load main dashboard if a valid session token exists. 4. Navigation Mounting: Render the persistent bottom navigation bar (dashboard, timetable, settings). 5. Overlay Initialization: Attach the Global Floating Action Button (FAB) for the AI Assistant to the root view.
Outputs	● Localized Main UI Shell. ● Active Navigation Stack. ● Visual Theme (Colors/Fonts).
Error Handling	● If the system language is not supported, automatically fallback to Turkish. ● If a screen fails to mount, reset the navigation stack to the dashboard.

- **Carbon Footprint Calculation & Display**

Name	Carbon Footprint Calculator
Purpose / Description	The system shall provide a carbon footprint calculation feature that estimates the agricultural carbon dioxide emission impact of a field based on farmer-provided inputs. The calculated footprint shall be associated with the corresponding user and field and presented to the user through the mobile application.
Inputs	● Farmer responses to predefined multiple-choice questions ● Optional numeric inputs related to energy, fuel, or fertilizer usage

Processing	1. UI Form Handling
	  ● Display a separate “Carbon Footprint” section in the mobile app.
	  ● Collect answers via multiple-choice and numeric fields.
	  ● Validate that required questions are answered before submission.

Name	**Carbon Footprint Calculator**
	2. Backend Validation ● Validate schema (required fields, allowed options, numeric ranges). ● Map multiple-choice selections to internal coefficients. 3. CO_2e Calculation ● Compute component emissions (example components): ○ Irrigation energy emissions (based on energy source + runtime/kWh/liters) ○ Fertilizer-related emissions (based on fertilizer level and/or kg) ● And then produce: ○ Total_CO2e (kg CO_2e) ○ Breakdown 4. Store a record in carbon_footprint_results table, which possible row such as: ● user_id, total_co2e, breakdown_json, timestamp 5. Return computed values to UI in a standardized JSON object for immediate display.
Outputs	● Total carbon footprint estimate (kg CO_2e) ● Component breakdown
Error Handling	● If some multiple-choice answers are missing, the UI prevents submission and highlights incomplete fields. ● If numeric inputs are out of range or invalid (negative, non-numeric), the backend rejects request with message. ● If emission-factor mapping fails, the backend defaults that component to unavailable and flags the response with a warning. ● If database write fails, the system retries automatically; on repeated failure, it returns the computed result to the user but marks it as “not saved” and logs it for admin review.

3.3. Data Preprocessing

3.3.1. Irrigation Data Preprocessing

The irrigation recommendation module relies on high-quality environmental and soil data.

To ensure model stability, accuracy, and reproducibility, the system shall perform a structured preprocessing pipeline on the **Mendeley Crop Recommendation Dataset (Crop Recommendation using Soil Properties and Weather Prediction)** before training and inference.

This dataset provides **optimal soil and weather conditions** associated with **successful crop growth**, making it suitable not only for crop-type classification but also for deriving **irrigation-related decision patterns**. Because the dataset reflects agronomically meaningful and validated ranges of pH, temperature, moisture-related indicators, and seasonal weather parameters, these values can be used as reference inputs for the system's irrigation recommendation engine.

- **Raw Data Normalization**

  Some dataset attributes contain inconsistent numeric formatting (e.g., values recorded as "1.222.333.444").
  The system shall normalize all numeric fields by:

  - detecting malformed values,
  - removing redundant thousand separators,
  - converting strings into valid floating-point representations.

  This ensures uniform numeric encoding before analytical operations.

- **Missing Value Analysis**

  The system shall compute NA ratios for all features and generate a missing-value summary report.
  This report is required before any imputation or cleaning steps.

- **Outlier Detection and Removal**

  Before performing NA imputation, the system shall apply the **IQR (Interquartile Range) method** to detect and remove out-of-range values in features prone to measurement noise, such as:

- pH
- T2M_MAX_Spring
- T2M_MIN_Autumn
- other temperature, moisture, or weather-derived attributes.

Purpose: Removing outliers prior to imputation prevents skewed statistics that could bias median/mean calculations.

- **Class Distribution Documentation**

  To support fairness and balance in model performance, the system shall compute and store the distribution of all crop types in the dataset.

  This includes documenting:

  - number of samples per crop category,
  - detection of underrepresented crops,

- **Statistical Future Documentation**

  The system shall compute and save descriptive statistics for each feature, including:

  - mean
  - median
  - standard deviation
  - range
  - potential seasonal deviations

  This documentation supports both explainability and validation of preprocessing decisions.

- **Missing Value Imputation**

  After outlier removal, the system shall impute missing values using an appropriate statistical or model-based method, selected according to feature characteristics:

  - Numeric environmental features shall be imputed using:
    - median imputation or
    - season-aware imputation, when applicable.

  - Derived or categorical fields shall be imputed using:
    - mode imputation or

    - predefined default values if the system design requires it.

- **Synthetic Sample Generation (Class Balancing)**

  For crop types with insufficient representation (e.g., Pepper and Potato), the system shall apply synthetic data augmentation techniques such as:

  - SMOTE or
  - domain-aware synthetic sampling,

  to ensure that minority classes are adequately represented during model training. This improves model fairness and prediction stability.

- **Feature Scaling**

  To ensure consistency across features with different numerical ranges:
  The system shall apply feature scaling using:

  - Min–Max scaling or
  - Standard Scaling

  The scaling method must be fixed and applied identically during inference.

- **Correlation and Feature Relationship Analysis**

  The system shall compute a correlation matrix for all numerical features to support feature selection and model interpretability.

  This analysis will be used to:

  - identify redundant features,
  - detect multicollinearity,
  - validate expected agronomic relationships (e.g., temperature vs. soil moisture trends).

- **Exploratory Data Analysis (Visual Validation)**

  To verify preprocessing steps and detect structural anomalies in the dataset, the system shall perform EDA visualization, such as:

  - distribution plots (histograms, KDEs),
  - boxplots for outlier evaluation,

- crop-type frequency charts,
- correlation heatmaps,
- seasonal trend visualizations.

These visualizations serve as quality-control artifacts and are not required at runtime but are mandatory for model-development documentation.

3.3.2. Image Data Preprocessing for Disease Detection

The disease detection module relies on accurate, noise-reduced, and consistently formatted image data to ensure model stability, high inference accuracy, and reproducible visual diagnostics. To support both training and inference, the system shall apply a structured preprocessing pipeline to all plant leaf images captured/uploaded through the TARAS application. This pipeline ensures that the CNN-based disease classifier operates on standardized, clean and meaningful visual inputs.

3.3.2.1. Raw Image Validation

The system shall validate the integrity and format of each incoming image before performing any analytical operation.

Validation includes:

- checking file format (JPEG/PNG),
- verifying that file size does not exceed predefined limits,
- ensuring that the image is not corrupted,
- confirming minimum acceptable resolution for classification.

Images failing any of these checks shall be rejected and logged for quality assurance.

3.3.2.2. Image Resizing and Aspect-Ratio Normalization

In order to maintain consistency across the CNN input pipeline, the system shall:

- resize all images to the model's expected input dimensions
- preserve aspect ratio when possible,
- apply center-cropping/padding to prevent distortion.

This ensures that all images conform to the numerical structure required by the model.

3.3.2.3. Color Space Standardization

To ensure consistent lighting interpretation and feature extraction, the system shall convert all images to a standardized color space.

Operations include:

- converting raw images to RGB,
- normalizing pixel intensity ranges to 0–1 (depending on model specification),

This enables reproducible visual feature extraction across devices.

3.3.2.4. Noise Reduction

Mobile field images often contain shadows, glare or compression artifacts. The preprocessing pipeline shall apply optional filters such as:

- Gaussian blur (low-intensity smoothing),
- median filtering for salt-and-pepper noise,
- contrast normalization for uneven lighting conditions.

These operations reduce noise while preserving disease-relevant texture patterns.

3.3.2.5. Leaf Region Isolation

The system performs leaf segmentation to isolate disease-relevant regions. Methods include:

- color-threshold masking (green-channel analysis),
- background removal,
- contour detection.

Isolating the leaf region reduces false positives caused by soil, sky, or human-made background elements.

3.3.2.6. Data Augmentation for Model Generalization

To improve model robustness and prevent overfitting, the training pipeline shall apply controlled augmentation techniques, such as:

- random rotations and flips,
- controlled brightness/contrast adjustments,
- slight zoom or crop transformations.

Augmentation must not distort disease-critical features.

3.3.2.7. Image Quality Assessment Checks

Before inference or training, the system shall evaluate image quality using criteria such as:

- blur detection (Laplacian variance),
- lighting adequacy,
- obstruction analysis (e.g., partially visible leaves).

Images failing minimum quality thresholds shall be flagged or optionally rejected.

3.3.2.8. Dataset Class Distribution Documentation

To support fairness and balanced classification accuracy, the system shall compute and store the distribution of disease classes in the annotated dataset. This documentation includes:

- number of images per disease category,
- identification of underrepresented diseases,
- recommendations for synthetic augmentation or targeted data collection.

3.3.2.9. Normalization for Inference

During inference, the system shall apply the same normalization parameters used during training, specifically:

- mean subtraction,
- standard deviation division,
- reordering to the model's expected tensor structure.

Consistency between training and inference pipelines is mandatory for reliable performance.

3.4. Non-Functional Requirements

3.4.1. Performance Requirements

Under normal operating conditions, sensor data is expected to become available to the backend and mobile application within **30 seconds to 1 minute** after collection. The mobile application is expected to respond to user interactions such as tab navigation, data loading and dashboard updates within **1–2 seconds**. Irrigation analysis and recommendation generation is expected to complete within a few seconds after relevant

data becomes available. Disease-detection results are expected to be delivered within **5–10 seconds** following image upload, depending on network conditions. In cases of limited or unstable connectivity, the system is expected to immediately display cached or last-known data to maintain usability.

3.4.2. Reliability Requirements

The system is expected to work reliably in real agricultural environments where network interruptions and temporary component issues may occur. Under normal conditions, sensor data is expected to be successfully delivered for around **95–98%** of the time and any missing data being limited to short gaps. If communication is temporarily unavailable, the system is expected to continue using the last known data for up to **10–30 minutes** until new measurements arrive. Once connectivity is restored, data synchronization between field devices and the backend is expected to resume automatically within a few minutes. Any data loss is expected to be minimal and occasional rather than continuous. In case of temporary failures in sensor nodes, gateways or backend services, the system is expected to keep operating in a reduced but stable state and return to normal operation automatically when the issue is resolved.

3.4.3. Availability Requirements

The system is expected to remain available for regular use throughout the agricultural season. Backend services are expected to achieve an overall availability of around **97–99%** under normal conditions. The application is expected to be accessible whenever the user's device is operational and core features remain available even during temporary outages. Planned maintenance or short service interruptions are expected to be infrequent and limited in duration, usually not exceeding a few minutes at a time. In the event of connectivity loss, the system is expected to continue providing access to previously synchronized data. Once connectivity is restored, the system is expected to resume normal operation automatically without requiring user intervention.

3.4.4. Scalability Requirements

The system is expected to scale gradually as the number of fields and deployed sensor nodes increases over time. In early deployments, the platform will start with a prototype setup in a limited field area, initially using two sensor nodes. The number of sensors is not fixed and is expected to be adjusted iteratively based on collected data, performance, and the quality of recommendations produced by the system. Data storage and processing components are expected to accommodate continuous growth across multiple growing seasons. Therefore, the overall data volume is expected to increase steadily as new

measurements and field records are accumulated, rather than increasing abruptly.

3.4.5. Security Requirements

The system is expected to protect user accounts and field related data from unauthorized access. Under normal operating conditions, only authenticated users shall be able to access the system. And each user is supposed to only see the information that they are allowed. User login information and other sensitive data are expected to be protected during both transmission and storage. Data related to fields, sensor outputs and analysis results are expected to be available only to the authorized users. Images and inputs uploaded by the users and inputs are expected to be secure in order to prevent data exposure. As the number of users increase, the system is expected to maintain this level of security without affecting usability and performance.

3.4.6. Usability Requirements

The system is expected to be usable by farmers with limited technical experience. Under normal operating conditions, a first time user is expected to understand and perform basic tasks like viewing field data or checking irrigation recommendations. Users should access most actions in the mobile app within 2 to 4 steps. Visual elements are expected to be readable under outdoor conditions and key information shall be distinguishable within three to five seconds of viewing. New users are expected to become familiar with the system's main features after two short tutorial sessions at most. As the system evolves, the user effort required to perform common tasks is expected to remain in the limits stated previously in this paragraph.

3.4.7. Maintainability Requirements

The system is expected to be maintainable with limited effort. Under normal operating conditions, common maintenance activities like configuration changes, minor bug fixes, or small updates are expected to be completed within 1 to 2 maintenance sessions with 10 to 30 minutes of down time at most. Software updates are supposed to be able to be deployed remotely. Also they shall not require more than one technician. When a system fault occurs, the cause of the problem should be identified in 30 to 60 minutes. Minor fixes are expected to be resolved in a work day, while more complex issues are expected to be resolved in 2 to 5 work days.

3.4.8. Portability Requirements

The system is supposed to be portable across commonly used platforms and environments. Under normal operating conditions, the mobile app is expected to run on both iOS and Android devices without platform specific

changes. Backend components are expected to be deployable across different server and cloud environments with one configuration adjustment. System migration to a new environment is expected to be completed in one work day without affecting stored data or functionality.

4. Analysis - UML

4.1. Use Cases

4.1.1. Use Case Diagram

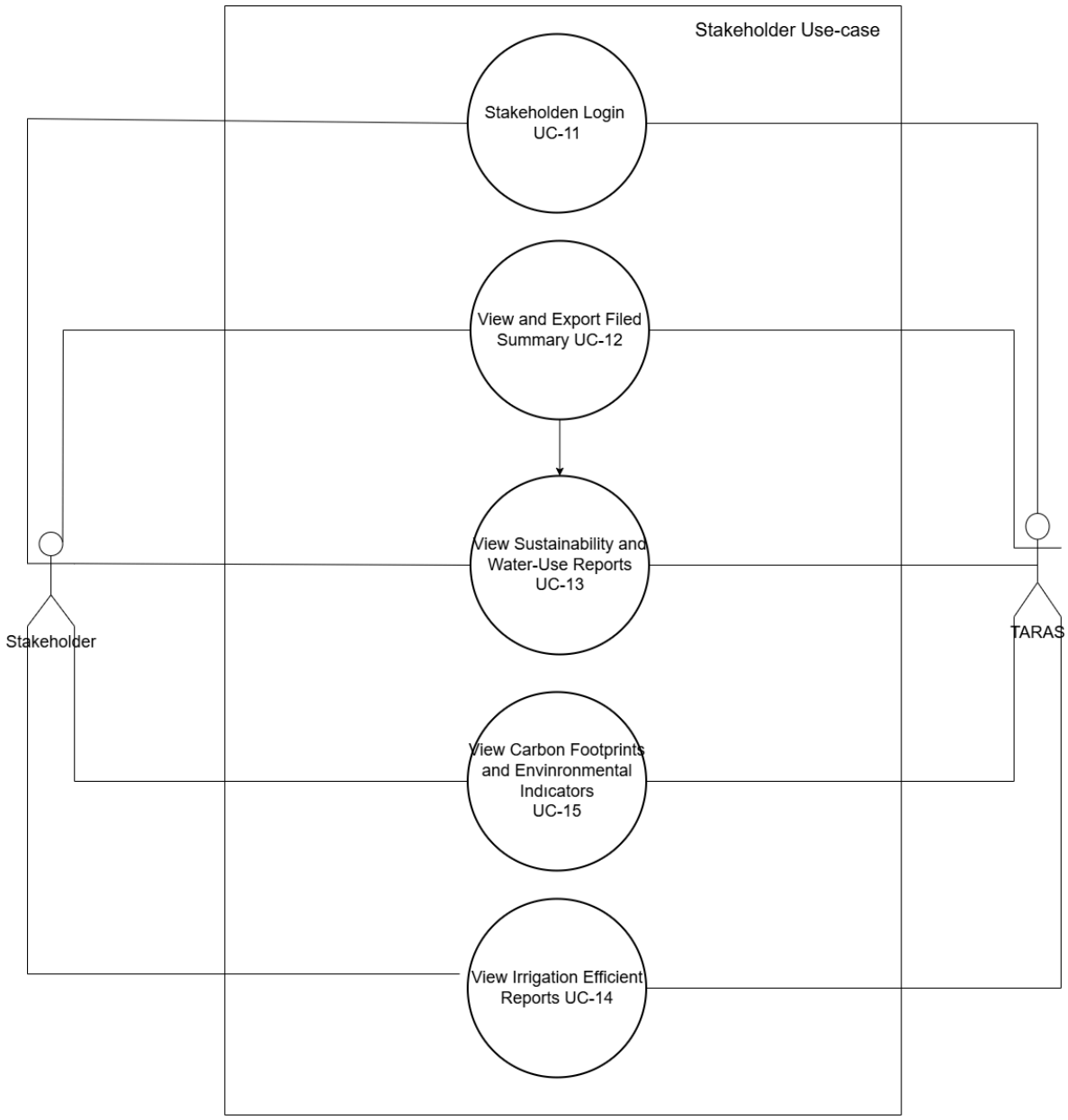


Diagram 1: Stakeholder Use Case Diagram

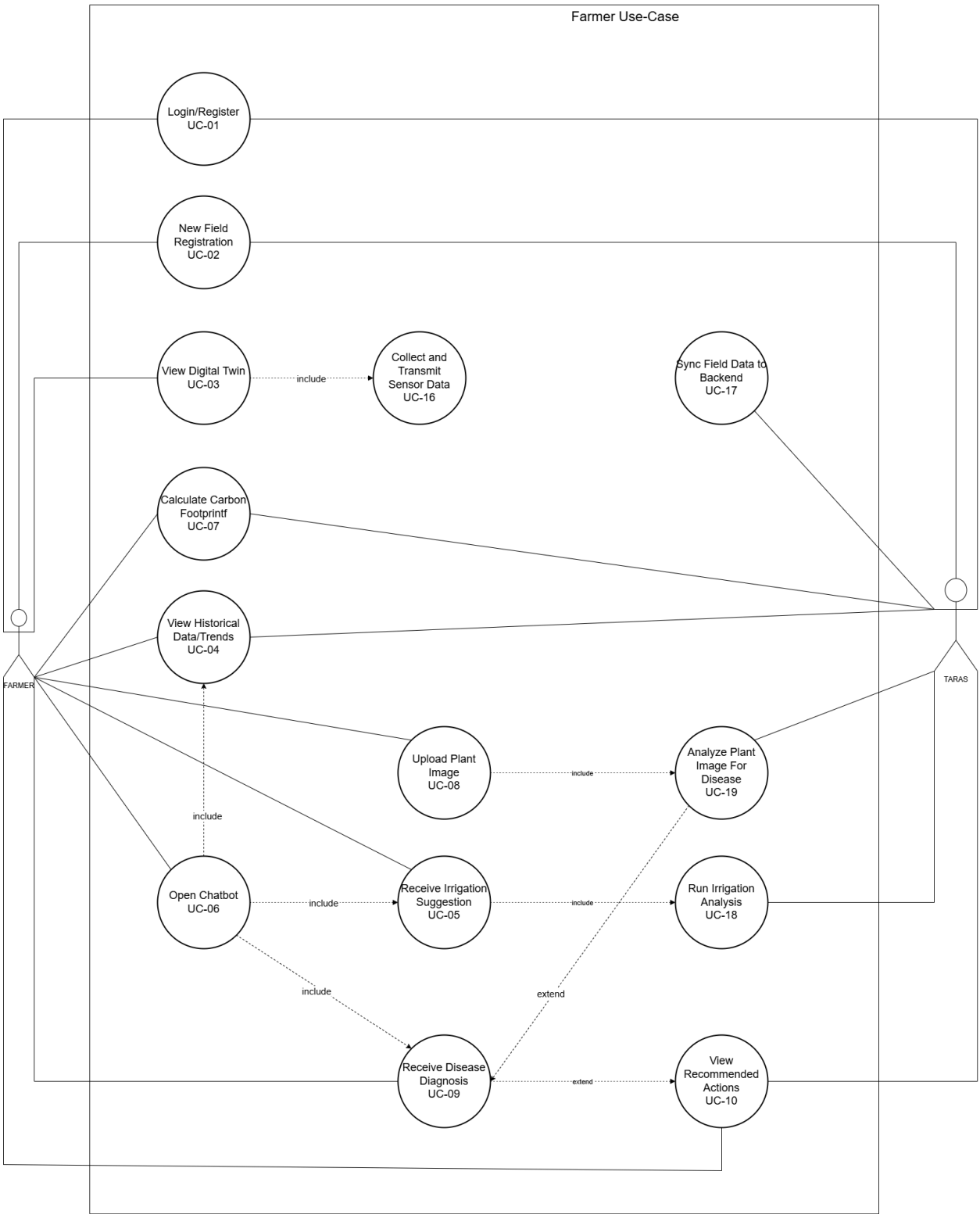


Diagram 2: Farmer Use Case Diagram

4.1.2. Description of Use Cases

Use Case Name	Login/Register
Use Case #	UC-01
Actors	Farmer
Description	The system manages user login and registration and grants access to the TARAS mobile application according to the user's role.
Precondition	● The user has access to the TARAS application installed. ● The user has a stable internet connection for the login/registration process. ● For login, the user already has a registered account.
Scenario	1. The system displays a login/register screen. 2. User chooses Login or Registration 3. For login: a. User inputs credentials b. System verifies information 4. For registration: a. User inputs required information b. The system validates the inputs and creates a new user account. 5. Upon successful authentication, the system loads the appropriate dashboard
Postcondition	● Users are authenticated and granted access to the system according to their role. ● The user session is initialized and stored for further operations.
Exceptions	● Incorrect username/password = System displays an error message. ● Weak or missing registration information = System rejects input and prompts correction. ● Network failure = System cannot authenticate and inform the user. ● Suspended or disabled account = System denies access.
Related Use Cases	

Use Case Name	New Field Registration
Use Case #	UC-02
Actors	Farmer
Description	The system records a new field for a registered farmer and links it to the selected sensor nodes so that incoming data can be associated with the correct field.
Precondition	● The farmer is already logged into the system. ● Sensor nodes are already in the field.
Scenario	● System scans for nearby sensor nodes/shows available node IDs. ● Farmer selects which sensor nodes belong to this field. ● The system links the selected node IDs to the new field. ● The system confirms that the field has been registered.
Postcondition	● A field record is created in the system. ● The selected sensor nodes are associated with this field, allowing incoming data to be grouped accordingly.
Exceptions	● No sensor nodes detected = System notifies the farmer. ● Invalid or duplicate IDs = System asks for correction. ● Network issues = System cannot complete the registration.
Related Use Cases	

Use Case Name	View Digital Twin
Use Case #	UC-03
Actors	Farmer
Description	Users view the real-time and historical moisture/temperature distribution of the field via digital twin.
Precondition	Sensor data exists in DB.
Scenario	1. The farmer opens the digital twin page. 2. The app sends requests for the latest sensor values. 3. Backend queries DB hypertables. 4. Aggregated & real-time values returned. 5. Heatmap rendered on device.
Postcondition	Digital Twin updated with fresh data.
Exceptions	● No recent sensor data available. ● The backend cannot access DB. ● Device offline = cached data shown.
Related Use Cases	UC-16

Use Case Name	View Historical Data/Trends
Use Case #	UC-04
Actors	Farmer
Description	The system displays historical information for a selected field over a chosen time range, helping the farmer understand past field conditions and the effects of irrigation decisions.
Precondition	● Farmer is logged in. ● At least one field is registered and linked to sensor nodes. ● Historical data exists in the system for the selected field (or at least some past measurements are stored).
Scenario	1. Farmer chooses the history/trends option in the app. 2. System displays a time-range selector 3. Farmer selects a time range and confirms. 4. The system retrieves historical sensor and/or irrigation data for that period. 5. The system displays graphs or charts (e.g., moisture vs. time, temperature vs. time, irrigation events on the timeline). 6. Farmers may scroll, zoom, or switch between variables (moisture, temperature, irrigation). 7. Farmer closes the view or returns to the field overview.
Postcondition	● The farmer has viewed historical trends for the selected field over the chosen period.
Exceptions	● No data available for the selected period = System shows an informative “No data for this period” message and suggests another range. ● Partial data gaps = System shows available data and may mark missing intervals. ● Network/connectivity issues = System cannot fetch new history and offers cached trends if available.
Related Use Cases	UC-06

Use Case Name	Receive Irrigation Suggestion
Use Case #	UC-05
Actors	Farmer
Description	The system generates irrigation recommendations for a logged-in farmer using recent sensor data and presents simple, actionable guidance such as when and how much to irrigate.
Precondition	● Farmer is logged in. ● At least one field is registered and linked to sensor nodes. ● The system has recent sensor data or last cached data for the selected field. ● ML/decision logic is available and configured on the backend.
Scenario	1. Farmer opens the TARAS application. 2. Farmer taps the “Irrigation Recommendation” button (or opens the field dashboard where recommendations are shown). 3. The system collects the latest available data for that field (e.g., soil moisture, temperature, crop type, growth stage, recent irrigation logs). 4. The system runs the irrigation decision logic/ML model using the collected data. 5. The system generates a simple, human-readable recommendation. 6. The farmer reviews the recommendation and decides whether to irrigate.
Postcondition	The farmer has received an updated irrigation recommendation for the selected field.
Exceptions	● No recent sensor data available = System informs the farmer (“No up-to-date data, using last known values” or “Recommendation cannot be generated”). ● ML model/service not reachable = System shows an error and suggests trying again later. ● Network/connectivity issues = System cannot request a fresh recommendation and may show the last stored recommendation if available.
Related Use Cases	UC-06,UC-18

Use Case Name	AI Decision-Support Chatbot
Use Case #	UC-06
Actors	Farmer
Description	The farmer interacts with a conversational AI assistant that uses a Large Language Model (LLM) to interpret sensor data, irrigation recommendations, historical trends, and disease-detection results. The chatbot provides natural-language explanations, guidance, and actionable decision-support suggestions tailored to the field's current conditions.
Precondition	● The user is authenticated and has access to the chatbot interface. ● Recent sensor data, irrigation recommendations, or disease analysis results are available. ● The LLM service is online and reachable by the backend.
Scenario	1. The user opens the AI assistant interface within the mobile application. 2. The user enters a question, command, or request for advice (e.g., *"Should I irrigate today?"* or *"What does this disease mean for my field?"*). 3. The system retrieves relevant contextual data: ● Latest sensor readings (soil moisture, temperature, humidity) ● Irrigation recommendations generated by the ML model ● Disease-detection results, if available ● Weather and historical trends (optional) 4. The backend compiles this data into a structured context package. 5. The LLM processes the user's question together with the contextual data. 6. The LLM generates a natural-language response with explanations or recommended actions. 7. The system returns the response to the user through the chat interface. 8. The user reads or follows the provided guidance.
Postcondition	● The farmer receives a personalized, context-aware advisory message. ● The conversation may be logged for future analytics or model improvement.
Exceptions	● The LLM service is unreachable or times out. ● No relevant sensor or irrigation data is available for the query. ● User query is ambiguous or cannot be interpreted. ● Backend fails to compile context data. ● Response generation error occurs (model failure).
Related Use Cases	UC-04, UC-05, UC-09

Use Case Name	Calculate Carbon Footprint
Use Case #	UC-07
Actors	Farmer
Description	The farmer requests a carbon footprint calculation for a selected field by providing basic activity-related inputs. The system uses this information together with existing field data to estimate the environmental impact and presents the result to the farmer [9].
Precondition	● Farmer is logged in. ● At least one field is registered and selected. ● The farmer has access to the carbon footprint feature.
Scenario	1. Farmer opens the “Carbon Footprint” or “Environmental Impact” section. 2. The system prompts the farmer to enter required inputs (e.g., irrigation method, energy source, usage level). 3. Farmer provides the requested inputs and confirms the calculation request. 4. The system processes the provided information together with existing field data. 5. The system calculates the estimated carbon footprint for the field. 6. The calculated result is displayed to the farmer in a clear and understandable format.
Postcondition	● A carbon footprint estimate is generated for the selected field. ● The result becomes available for viewing and comparison over time.
Exceptions	● Required inputs are missing = System asks the farmer to complete the information. ● Calculation cannot be completed = System informs the farmer that the result is temporarily unavailable.
Related Use Cases	

Use Case Name	Upload Plant Image
Use Case #	UC-08
Actors	Farmer
Description	The user selects or captures a plant leaf image and uploads it to the system for disease analysis.
Precondition	● The user is authenticated. ● The device has network connectivity/queued upload mode enabled.
Scenario	1. The user navigates to the disease detection page. 2. The user captures an image or selects an existing image. 3. The system validates the selected image type and size. 4. The image is uploaded to the backend for further processing. 5. The system acknowledges successful upload.
Postcondition	The image is received and stored temporarily by the backend for analysis.
Exceptions	● Invalid or unsupported file format. ● Image corrupted or unreadable. ● Network failure during upload. ● Upload exceeds size limit.
Related Use Cases	UC-19

Use Case Name	Receive Disease Diagnosis
Use Case #	UC-09
Actors	Farmer
Description	After uploading an image, the user receives the disease classification, confidence score, and severity returned by the system.
Precondition	● A valid image has been uploaded. ● The inference engine is available.
Scenario	1. The system preprocesses the uploaded image. 2. The system runs the disease-detection model on the image. 3. The model generates a label, severity level, and confidence score. 4. The backend sends the diagnosis result to the user's device. 5. The user views the diagnosis.
Postcondition	● Diagnosis is displayed to the user. ● The result is stored in the database
Exceptions	● Model service unavailable. ● Inference timeout. ● Disease cannot be identified with sufficient confidence. ● Backend failure returning the result.
Related Use Cases	UC-06,UC-10, UC-19

Use Case Name	View Recommended Actions
Use Case #	UC-10
Actors	Farmer
Description	The user views actionable recommendations generated based on the diagnosed disease (e.g., treatment steps, risk warnings).
Precondition	● A diagnosis has been successfully generated ● Recommendation rules or mappings exist for the detected disease.
Scenario	1. The system matches the diagnosed disease to predefined recommendation rules. 2. The system generates appropriate treatment or monitoring actions. 3. The system presents the recommended actions on the app. 4. The user reviews the suggested next steps.
Postcondition	● Users receive actionable treatment guidance.
Exceptions	● No recommendation exists for the specific disease. ● Backend fails to retrieve recommendation rules. ● User interface fails to load the recommendation section.
Related Use Cases	UC-09

Use Case Name	Stakeholder Login to Dashboard
Use Case #	UC-11
Actors	Stakeholder
Description	A stakeholder logs into the platform to access high-level summaries, sustainability metrics, and environmental reports for the agricultural fields they have a stake in.
Precondition	● The stakeholder has a valid account and credentials. ● System authentication services are available.
Scenario	1. Stakeholder opens the dashboard login page. 2. Stakeholder enters email/username and password. 3. System verifies credentials. 4. The system loads the stakeholder dashboard with authorized field data. 5. Stakeholders gain access to summary and reporting tools.
Postcondition	● Stakeholders are successfully authenticated and connected to their dashboard.
Exceptions	● Invalid credentials. ● User account disabled or missing necessary permissions. ● Authentication service unavailable. ● Network failure.
Related Use Cases	

Use Case Name	View and Export Field Summary (Period-Based)
Use Case #	UC-12
Actors	Stakeholder
Description	The stakeholder views aggregated summaries for selected periods (year, month, quarter) and may export these summaries for reporting or compliance purposes.
Precondition	● Stakeholder is logged in ● Relevant historical data exists in the database.
Scenario	1. Stakeholders select a time period (year/month/custom). 2. The system retrieves aggregated sustainability, irrigation, and efficiency data for the selected period. 3. The system displays the summarized metrics. 4. Stakeholders optionally choose to export the summary. 5. The system generates a downloadable file (PDF, CSV, or Excel).
Postcondition	● Summary is displayed and, if requested, exported successfully.
Exceptions	● No data available for the selected period. ● Export generation failure. ● Backend query timeout.
Related Use Cases	

Use Case Name	View Sustainability and Water-Usage Reports
Use Case #	UC-13
Actors	Stakeholder
Description	Stakeholders view reports on water consumption patterns, sustainability indicators, and long-term performance of the fields they oversee.
Precondition	● Stakeholder is logged into the dashboard. ● Sustainability-related metrics are available in the database.
Scenario	1. Stakeholder navigates to the *Sustainability Reports* section. 2. The system retrieves water-usage history, consumption trends, and sustainability metrics. 3. The system displays charts, tables, and key performance indicators. 4. Stakeholder reviews the sustainability performance of the selected fields. M
Postcondition	● Stakeholders gain insight into sustainability and resource-efficiency performance.
Exceptions	● Missing or incomplete sustainability data. ● Report generation service unavailable. ● Database query failure.
Related Use Cases	

Use Case Name	View Irrigation Efficiency Reports
Use Case #	UC-14
Actors	Stakeholder
Description	Stakeholders examine irrigation efficiency metrics to evaluate whether fields are overwatered, underwatered, or operating optimally.
Precondition	● The stakeholder is authenticated. ● Irrigation logs and sensor data are available.
Scenario	1. Stakeholder selects the field or region to analyze. 2. The system retrieves irrigation events and moisture response data. 3. The system computes irrigation efficiency indicators. 4. The system presents graphs and efficiency summaries. 5. Stakeholders interpret the field's irrigation performance.
Postcondition	● Stakeholders view efficiency insights for better oversight and compliance.
Exceptions	● Irrigation data missing or corrupted. ● Efficiency metrics cannot be computed due to insufficient data. Backend analytics service failure.
Related Use Cases	

Use Case Name	View Carbon Footprint and Environmental Indicators
Use Case #	UC-15
Actors	Stakeholder
Description	Stakeholders view carbon-footprint calculations and environmental indicators such as fuel usage, energy efficiency, runoff risk, and sustainability scores.
Precondition	● Carbon-footprint-related inputs exist (fuel logs, irrigation energy usage, fertilizer use, etc.). ● Stakeholders are logged into the system.
Scenario	1. Stakeholder navigates to the Environmental Indicators section. 2. The system retrieves carbon footprint calculations and environmental KPIs. 3. The system generates charts and detailed metrics. 4. Stakeholders review environmental performance of all relevant fields.
Postcondition	● Environmental and carbon-impact metrics are presented clearly to the stakeholder.
Exceptions	● Missing or incomplete environmental data. ● Carbon footprint model service unavailable. ● Query timeout or calculation failure.
Related Use Cases	

Use Case Name	Collect and Transmit Sensor Data
Use Case #	UC-16
Actors	System
Description	Sensor nodes placed in the field automatically collect environmental measurements and send them to the TARAS system through the gateway, where the data is stored and used for monitoring, digital twin updates, and irrigation recommendations.
Precondition	● The sensor node is powered on and calibrated. ● Node is correctly paired with a field and registered in the system. ● LoRa gateway is operational and connected to the backend. ● Sampling interval is configured
Scenario	1. The sensor node collects environmental measurements in the field. 2. The node prepares these measurements as a single data packet. 3. The node wirelessly sends this packet to the gateway. 4. The gateway forwards the received data to the backend system over the internet. 5. The backend checks the data, verifies which field it belongs to and accepts it if valid. 6. The backend saves the new data, updates the field's latest status. 7. The system automatically triggers any follow-up processes (digital twin update, trend update, irrigation model input).
Postcondition	● New sensor data is stored in the database and available for farmer dashboards. ● Digital twin and field status are updated based on the latest measurements.
Exceptions	● If the system does not receive new data, it keeps showing the last available information. ● The system may notify the farmer if data has not been received for an unusually long time.
Related Use Cases	UC-03

Use Case Name	Sync Field Data to Backend
Use Case #	UC-17
Actors	System
Description	Field data collected in the field, including sensor readings and irrigation logs, is synchronized to the database so that all information is safely stored and kept up to date.
Precondition	● Sensor nodes and/or the mobile app have collected new field data. ● At least one field is registered in the system. ● There is at least temporary internet connectivity at the gateway or mobile device
Scenario	1. New field data becomes available on the gateway or mobile app 2. The system prepares this data to be sent to the server. 3. The system sends the data to the backend when an internet connection is available. 4. The backend receives the data and saves it in the database. 5. The records for the related field are updated. 6. Updated information becomes available for farmer dashboards, digital twin, history, and recommendations.
Postcondition	● Field data stored on local devices is synchronized with the central TARAS system. ● The backend holds the latest version of field information.
Exceptions	● If no internet connection is available, the system keeps the data locally and retries later. ● If synchronization fails repeatedly, the system may show a warning.
Related Use Cases	

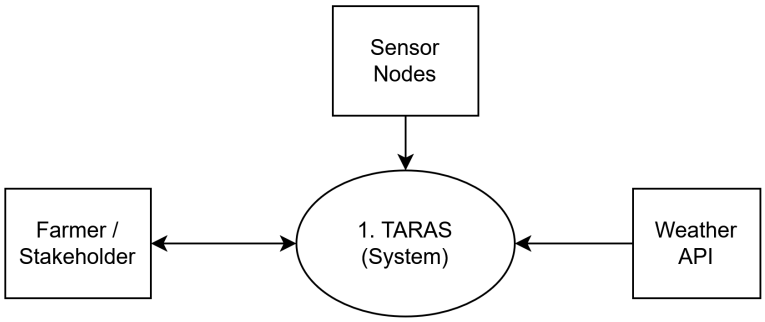
Use Case Name	Run Irrigation Analysis
Use Case #	UC-18
Actors	System
Description	The system performs irrigation analysis for each field by evaluating recent sensor data, historical irrigation records, and field-related information. The analysis results are stored and later presented to the farmer through visualization and recommendation interfaces.
Precondition	● A field is registered and linked to sensor nodes. ● Sensor and historical data are available in the system.
Scenario	1. New sensor data or relevant field updates become available. 2. The system gathers the required field data. 3. The system evaluates field conditions using the irrigation analysis logic. 4. An irrigation recommendation and related indicators are generated. 5. The results are stored and made available for visualization.
Postcondition	● Updated irrigation analysis results are available for the field.
Exceptions	● If required data is missing, the system delays analysis or uses previously available information. ● If the analysis process cannot be completed, the system marks the result as unavailable.
Related Use Cases	UC-05

Use Case Name	Analyze Plant Image for Diseases
Use Case #	UC-19
Actors	System (Inference Engine, Backend Services)
Description	The system automatically processes an incoming plant image, runs the CNN model to identify potential diseases, and stores the result for retrieval by the user.
Precondition	● A valid image upload request has been received from the mobile client. ● The CNN model is available and loaded on the server.
Scenario	1. The system receives the uploaded leaf image. 2. The system performs validation(format, resolution, corruption check). 3. The image is preprocessed (resize, normalization). 4. The system executes CNN inference to detect disease type/severity. 5. The system generates confidence scores and optional explanation maps. 6. The system stores the classification result in the database. 7. The system returns the structured result to the mobile application.
Postcondition	● The image analysis result is stored in the disease_results table. ● The result becomes available for display in the disease detection module.
Exceptions	● Model service unavailable or inference timeout. ● Image preprocessing failure. ● Database insertion error. ● Unrecognized disease or insufficient confidence.
Related Use Cases	UC-08, UC-09

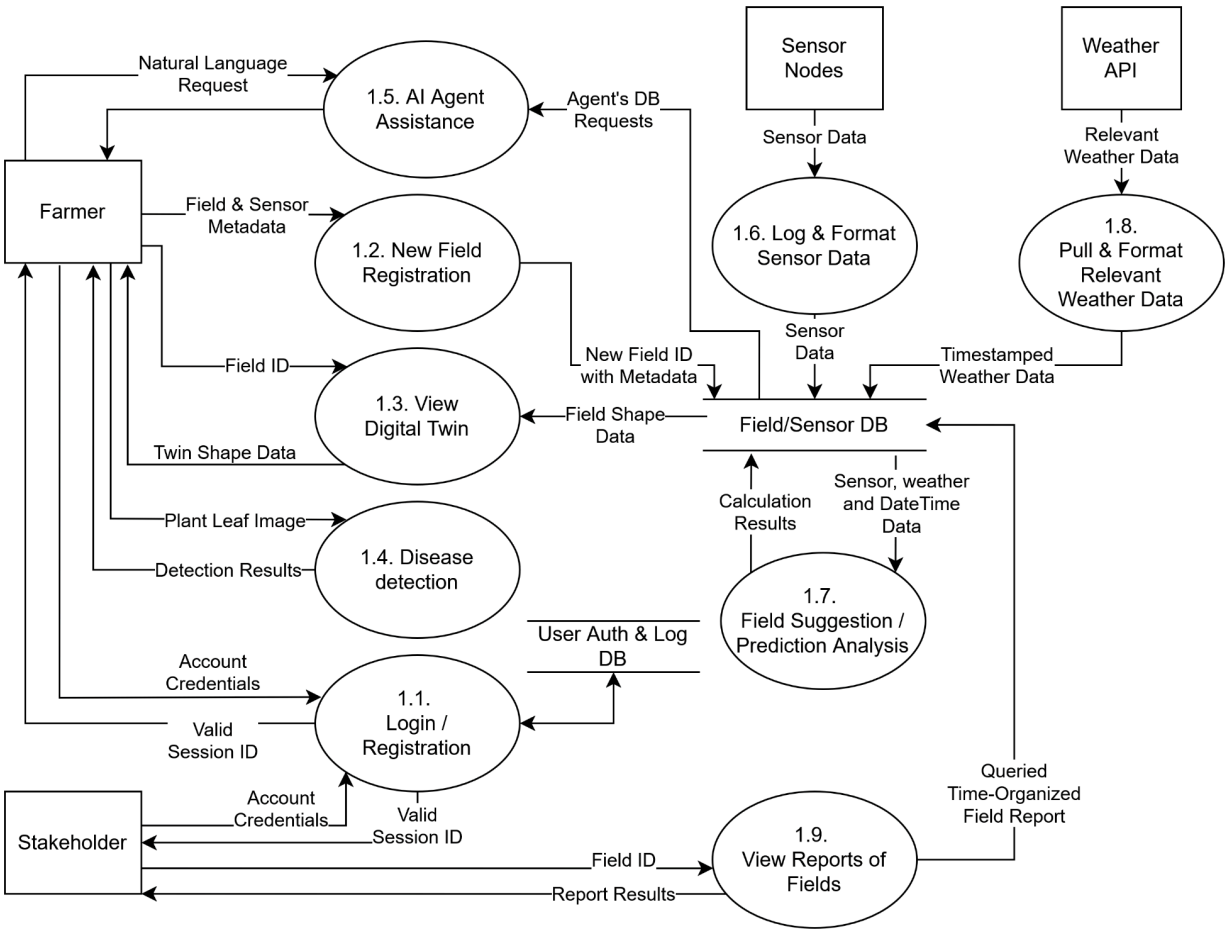
4.2. Functional Modelling(DFD)

4.2.1. DFD Diagrams

- **Level 0 - Context Diagram**



- **Level 1 DFD**



5. Conclusion

This Software Requirements Specification (SRS) successfully defines the complete architectural and functional scope of TARAS, establishing a solid foundation for the implementation of the system. Throughout this document, we have detailed how the platform will bridge the gap between complex data analysis and practical agricultural management, ultimately aiming to help farmers make better, data-driven decisions regarding irrigation and plant health.

The proposed design effectively integrates diverse technologies, from low-level IoT sensor nodes and LoRa communication to high-level machine learning algorithms and digital twin visualizations. A significant portion of this analysis focused on addressing the specific constraints of the agricultural domain. By defining rigorous requirements for store-and-forward mechanisms and offline tolerance, the system is designed to remain reliable even in rural environments where connectivity is often unstable.

In conclusion, this SRS ensures that all stakeholders have a shared technical understanding of the project. It balances advanced capabilities, such as the LLM-based advisory system and disease detection, with necessary non-functional requirements like usability and security. This document now serves as the comprehensive blueprint for developing TARAS into a scalable, user-friendly, and environmentally conscious precision agriculture tool.

6. References

[1] Espressif Systems, “ESP32-C6 Technical Reference Manual,” 2023.
[2] Sensirion AG, “SHT85 Datasheet,” 2022
[3] LoRa Alliance, “LoRaWAN Specification,” 2020.
[4] PostgreSQL Global Development Group, PostgreSQL Documentation
[5] Timescale Inc., “TimescaleDB Documentation – Continuous Aggregates,” 2024.
[6] PostGIS Project, PostGIS Manual
[7] W3C, “Web Content Accessibility Guidelines (WCAG) 2.1 - Success Criterion 1.4.3: Contrast (Minimum),” W3C Recommendation, 2018.
[8] W3C, “Web Content Accessibility Guidelines (WCAG) 2.1 - Success Criterion 2.5.5: Target Size,” W3C Recommendation, 2018.
[9] ISO, ISO 14064-1: Greenhouse Gases - Carbon Footprint Quantification, International Organization for Standardization, 2018.